Laura Hietala **Näyttötehtävä 4**

Tietojenkäsittelyn koulutus
TKMI23KV 6.5.2023

Ohjelmoinnin perustaidot
Janne Turunen

Tehtäväraportti

Tässä tehtäväraportissa esittelen tekstin ja kuvankaappausten keinoin, kuinka olen toteuttanut kurssilla annetut ohjelmointitehtävät. Käytetty ohjelmointikieli on kurssin vaatimusten mukaisesti TypeScript.

Osatehtävä A

Tee funktio **tarkastaPostinumero()**, joka tarkastaa suomalaisen postinumeron muodon oikeellisuuden. Jos funktiokutsun yhteydessä argumenttina annettu merkkijono (huom!) on kelvollinen suomalainen postinumero, palauttaa funktio totuusarvon true. Muussa tapauksessa palautetaan false.

Suomalainen postinumero koostuu viidestä numerosta 0-9. Postinumerot voivat siis teoriassa olla väliltä 00000 - 99999. Ohjelmoinnin kontekstissa postinumero ei ole numero tai luku (number), vaan merkkijono, joka koostuu numeroista (ja ei muista merkeistä). Kymmenjärjestelmässä number-tyyppinen arvo ei voi alkaa nollalla ilman sitä seuraavaa desimaalipistettä.

Funktion toteutuksessa ei tarvitse huomioida:

- Kansainvälisen postiliikenteen FI-etuliitettä. Voidaan olettaa, että postinumero koostuu ainoastaan viidestä numerosta.
- Postinumeroa vastaavan postitoimipaikan oikeaa olemassaoloa. Pelkkä muodon tarkastus riittää.

Lisäksi funktion toteutuksella on yksi rajoite. Ratkaisuun ei saa käyttää säännöllisiä lausekkeita (RegExp), sillä nämä eivät kuulu kurssin sisältöihin.

Testaa funktiota kutsumalla sitä argumenteilla "50100", "00100", "123456", "K54-92", "8454" ja "1.223". Kaksi ensimmäistä ovat kelvollisia (*true*) ja loppujen kohdalla funktion tulee palauttaa *false*. Raportoi testaus.

```
1   console.clear();
2
3   const tarkastaPostinumero = (postinumero: string): boolean => {
4     if (postinumero.length !== 5) return false;
5     for (let i = 0; i < postinumero.length; i++) {
6       const numero = postinumero.charCodeAt(i);
7       if (numero < 48 || numero > 57) return false;
8     }
9     return true;
10  };
11
12  console.log(tarkastaPostinumero("19233"));
```

- Rivi 1: Tyhjennetään Terminal-ikkuna console.clear()-metodilla.
- Rivit 3-10: Tehdään nuolifunktio ”tarkastaPostinumero(), jonka parametri ”postinumero” on string-tyyppiä. Funktio on palauttaa boolean-arvon. Rivillä 4 määritellään, että jos postinumeron pituus ei ole tasan 5, funktio palauttaa ”false”. Rivillä 5 on for loop, joka käynnistyy, jos rivi 4 ei palauta false. Silmukka tarkistaa jokaisen postinumeron numeron, eli 5 kappaletta numeroita. Jokaisessa silmukan iteraatiossa funktio käyttää charCodeAt() -metodia (rivillä 6) palauttaen Unicode-numeron merkille annetussa merkkijonossa. Tämä tallennetaan numero-muuttujaan. Rivin 7 ehto tarkistaa, onko numero-muuttujan arvo alle 48 tai yli 57 (näiden välissä on ASCII-koodit numeroille 0-9), ja jos näin on, palauttaa funktio arvon false. Rivillä 9 pyydetään funktiota palauttamaan arvon true, jos kaikki aiemmat ehdot ovat tosia, mikä tarkoittaa, että postinumero on validi. Koodilohkon suoritus päättyy aaltosulkeeseen rivillä 10.
- Rivi 12: Kutsutaan funktiota tarkastaPostinumero ja annetaan sille tarvittava parametri, eli merkkijono, jonka validius postinumeroksi halutaan tarkistaa. Testattava 19233 palautti arvon true, kuten kuuluukin. Testasin myös opettajan vaatimat kuusi merkkijonoa, joista kaksi ensimmäistä palauttivat ”true” ja neljä viimeistä ”false”

Osatehtävä B

Tee funktio nimeltään **milloinPukkiTulee()**, joka palauttaa tekstin ohjelman suorituspäivään perustuen joko:

- Joulupukki tulee xx päivän kuluttua
- Joulupukki tulee huomenna
  tai
- Joulupukki tulee tänään!

Oletetaan, että pukki vierailee jouluaattona. Muista ottaa huomioon jouluaaton jälkeiset päivät kuluvana vuonna (ei negatiivisia lukuja päivissä)!

Raportoi (ja kokeile) myös, kuinka voit testata funktiota ilman, että odotat jouluaattoon saakka.

```
console.clear();

const milloinPukkiTulee = () : string => {
  const joulu : any = new Date(new Date().getFullYear(), 11, 24);
  const tanaan : any = new Date();
  const msPaivassa : number = 24 * 60 * 60 * 1000;
  const paiviaJouluun : number = Math.floor((joulu - tanaan) / msPaivassa);

  if (paiviaJouluun > 1) {
    return `Joulupukki tulee ${paiviaJouluun} päivän kuluttua`;
  } else if (paiviaJouluun === 1) {
    return 'Joulupukki tulee huomenna';
  } else if (paiviaJouluun === 0) {
    return 'Joulupukki tulee tänään!';
  } else {
    return 'Joulupukki kävi jo.';
  }
};

const milloinPukkiTulisi = () : string => {
  const joulu : any = new Date(new Date().getFullYear(), 11, 24);
  const testipaiva : Date = new Date(2023, 11, 25);
  const msPaivassa : number = 24 * 60 * 60 * 1000;
  const testipaivastaJouluun = Math.floor((joulu.getTime() - testipaiva.getTime()) / (msPaivassa));

  if (testipaivastaJouluun > 1) {
    return `Joulupukki tulee ${testipaivastaJouluun} päivän kuluttua`;
  } else if (testipaivastaJouluun === 1) {
    return 'Joulupukki tulee huomenna';
  } else if (testipaivastaJouluun === 0) {
    return 'Joulupukki tulee tänään!';
  } else {
    return 'Joulupukki kävi jo.';
  }
};

console.log(milloinPukkiTulisi());
console.log(milloinPukkiTulee());
```

- Rivi 1: Tyhjennetään Terminal-ikkuna console.clear()-metodilla
- Rivi 3: Aloitetaan nuolifunktio nimeltään milloinPukkiTulee. Funktio palauttaa merkkijonon.
- Rivi 4: Funktio luo joulu-muuttujan, joka on uusi päivämääräobjekti: kuluvan vuoden 24. joulukuuta. Kuluva vuosi saadaan käyttämällä .getFullYear-metodia.
- Rivi 5: Funktio luo tanaan-muuttujan, joka on uusi päivämääräobjekti (nykyhetki).
- Rivi 6: Funktio luo msPaivassa-muuttujan, joka laskee millisekuntien määrän yhdessä vuorokaudessa.
- Rivi 7: Funktio luo paiviaJouluun-muuttujan, joka laskee, paljonko päiviä jouluun on nykyhetkestä, Math.floor() -metodi pyöristää tuloksen alaspäin lähimpään kokonaislukuun.
- Rivit 9-17: Annetaan funktiolle ohjeeksi palauttaa eri merkkijonoja sen mukaan, montako päivää jouluun on. Ehtolausekkeet palauttavat eri merkkijonon sen mukaan, onko jouluun aikaa >1 päivä (tällöin palautettava teksti pitää sisällään template stringin, joka kertoo tarkan päivien määrän jouluun nykyhetkestä alkaen), tasan 1 päivä, tasan 0 päivää tai jos mikään näistä ehdoista ei toteudu, funktio palauttaa ”Joulupukki kävi jo”.
- Rivit 20-24: milloinPukkiTulisi-funktio toimii periaatteessa samoin kuin milloinPukkiTulee-funktio, mutta päivät lasketaan erilailla. Nyt nykyisen päivämäärän sijaan luodaan muuttuja testipaiva, joka tässä esimerkissä sisältää päivämäärän 25.12.2023. Tätä päivämäärää voi muutella sen mukaan, minkä päivän ja joulun välisten päivien määrän erotuksen haluaa tarkistaa, ja testipaivastaJouluun-muuttujassa lasketaan tämä erotus.
- Rivit 26-34: Tarkistetaan päivien määrä samalla tavalla kuin ensimmäisessä funktiossa if…else -lauseiden avulla.
- Rivit 37-38: Kutsutaan kumpaakin funktiota erikseen omissa console.log() -metodeissaan.

Tämän jälkeen terminaaliin tulostui tämä:

